

A* Algorithm

Topic: Pathfinding and Path Planning using Python

Focus: Algorithm, heuristic, implementation, complexity, and practical applications

Introduction

A* is a graph-search and pathfinding algorithm used to find a low-cost path between a starting node and a goal node. It is widely used in robotics, games, maps, navigation systems, and autonomous systems.

The main idea behind A* is to combine two pieces of information: the cost already spent to reach the current node and an estimate of the remaining cost to reach the goal. This allows A* to focus its search toward the destination instead of exploring the entire search space.

Basic Terminology

A **node** represents a position or state. An **edge** represents a possible movement between two nodes. The **start node** is where the agent begins, and the **goal node** is the destination. An **obstacle** is a location that cannot be traversed.

The **path cost** is the total cost of the movements made along a path. A **heuristic** is an estimate of the remaining cost from the current node to the goal.

The Core Formula

A* assigns every candidate node an evaluation value:

$$f(n) = g(n) + h(n)$$

Here, **g(n)** is the actual cost from the start node to node n, **h(n)** is the estimated cost from n to the goal, and **f(n)** is the estimated total cost of a path through n.

At each step, A* generally selects the node with the smallest **f(n)** value for further exploration.

Understanding g(n), h(n), and f(n)

g(n): Measures what has already happened. It is the known cost from the start to the current node.

h(n): Predicts what remains. It estimates the cost from the current node to the goal.

f(n): Combines both values and represents the estimated total cost of a solution path passing through n.

Heuristic Functions

The quality of the heuristic is important because it influences how efficiently A* searches the space. A useful heuristic should estimate the remaining cost without overestimating it when optimality is required.

For a four-direction grid where movement is allowed only up, down, left, and right, a common heuristic is Manhattan distance:

$$h(n) = |x - x_{\text{goal}}| + |y - y_{\text{goal}}|$$

For a grid that permits diagonal movement with suitable movement costs, other heuristics such as Euclidean or Chebyshev distance may be appropriate.

How A* Works Step by Step

1. Put the start node into an open set (priority queue).
2. Give the start node a cost $g = 0$ and calculate its f value.
3. Remove the node with the lowest f value from the open set.
4. If it is the goal, stop the search and reconstruct the path.
5. Otherwise, inspect its valid neighboring nodes.
6. Calculate a tentative cost for reaching each neighbor.
7. If the new route is better than the neighbor's previously known route, update its cost and parent.
8. Add or update the neighbor in the priority queue.
9. Repeat until the goal is reached or no nodes remain.

Open Set and Closed Set

The **open set** contains nodes that have been discovered but still need to be evaluated. In practical implementations it is often represented using a priority queue or min-heap so that the node with the lowest priority can be selected efficiently.

The **closed set** contains nodes that have already been processed. Keeping track of visited nodes prevents unnecessary repeated work.

Path Reconstruction

A* normally stores a parent (or predecessor) for each node whenever a better route to that node is found. After reaching the goal, the algorithm follows parent references backward from the goal to the start. Reversing that sequence gives the final path from start to goal.

Simple Grid Example

Consider a rectangular grid containing free cells and blocked cells. The agent starts at one cell and must reach a goal while avoiding blocked cells. Each horizontal or vertical move can have a cost of 1.

For example, if the current cell is (2, 3) and the goal is (7, 8), the Manhattan heuristic is $|2-7| + |3-8| = 10$. The algorithm combines this estimate with the known cost $g(n)$ to calculate $f(n)$.

Python Implementation

The following implementation demonstrates A* on a four-direction grid. It uses Python's `heapq` module as a priority queue and Manhattan distance as the heuristic.

Example Code

```
import heapq

def heuristic(a, b):
    # Manhattan distance
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar(grid, start, goal):
    rows = len(grid)
    cols = len(grid[0])

    open_heap = []
    heapq.heappush(open_heap, (0, start))

    came_from = {}
    g_score = {start: 0}

    directions = [
        (-1, 0), # up
        (1, 0),  # down
        (0, -1), # left
        (0, 1)   # right
    ]

    while open_heap:
        current_f, current = heapq.heappop(open_heap)

        if current == goal:
            path = [current]

            while current in came_from:
                current = came_from[current]
            path.append(current)

            path.reverse()
            return path

        for dr, dc in directions:
            nr = current[0] + dr
            nc = current[1] + dc

            if not (0 <= nr < rows and 0 <= nc < cols):
                continue

            if grid[nr][nc] == 1:
                continue

            neighbor = (nr, nc)
            tentative_g = g_score[current] + 1

            if tentative_g < g_score.get(neighbor, float("inf")):
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g

            f_score = tentative_g + heuristic(neighbor, goal)
            heapq.heappush(open_heap, (f_score, neighbor))

    return None

grid = [
    [0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0, 0],
    [0, 0, 0, 1, 0, 0],
    [0, 1, 0, 0, 0, 0],
    [0, 1, 0, 1, 1, 0],
    [0, 0, 0, 0, 0, 0]
```

```

]

start = (0, 0)
goal = (5, 5)

path = astar(grid, start, goal)

if path:
    print("Path found:")
    print(path)
    print("Path length:", len(path) - 1)
else:
    print("No path exists.")

```

Code Explanation

heapq: Provides a min-heap priority queue.

heuristic(): Calculates Manhattan distance between a node and the goal.

neighbors(): Generates valid up, down, left, and right neighboring cells.

g_score: Stores the best known cost from the start to each cell.

came_from: Stores the parent of each cell for path reconstruction.

open_heap: Stores candidates ordered by their estimated total cost.

The algorithm returns the reconstructed path when the goal is reached. If the search finishes without reaching the goal, there is no path under the given movement and obstacle rules.

Correctness and Optimality

A* can find an optimal lowest-cost path when the heuristic is admissible, meaning it does not overestimate the true remaining cost. In graph-search implementations, additional consistency conditions can simplify the handling of already expanded nodes.

For a four-direction unit-cost grid, Manhattan distance is a standard admissible heuristic because it represents the minimum number of horizontal and vertical moves needed when obstacles are ignored.

Time and Space Complexity

The exact runtime of A* depends on the graph, heuristic, edge costs, and implementation. In the worst case, A* can require substantial time and memory because it may explore many nodes.

A priority queue improves practical performance compared with repeatedly scanning the entire open set. The algorithm can still use considerable memory because it stores discovered nodes, costs, and parent information.